

TraceCluster: A Lightweight and Adaptive Clustering-Based Subgraph Attention Network for APT Detection in Provenance Graphs

Lijuan Xu[✉], Zicheng Zhao[✉], Dawei Zhao[✉], Zhen Wang[✉], *Fellow, IEEE*, and Chunpeng Ge[✉]

Abstract—Provenance graph-based anomaly detection, particularly for Advanced Persistent Threat (APT) detection, addresses the issues of large-scale graphs and data imbalance. However, existing methods struggle with information loss, high computational complexity, and low detection accuracy. To address the above challenges, this paper proposes TraceCluster, a lightweight and adaptive clustering-based Subgraph Attention Network (SAN) for APT detection in provenance graph. TraceCluster mitigates the neighborhood explosion problem by clustering nodes to partition large-scale graphs, thus reducing reliance on the global graph while preserving local neighborhood information. Furthermore, the method dynamically models complex inter-node dependencies within subgraphs. It employs an attention mechanism to adaptively highlight the most relevant connections. This enhances node representations and improves overall feature extraction. This design substantially reduces memory consumption and avoids the high computational complexity of global graph processing. In addition, an adaptive category-weighting loss function assigns variable weights to different classes, improving the detection of rare and anomalous behaviors. Experimental results show that on the OpTC dataset, the currently faster method is 37-fold and 3-fold slower than our approach in terms of inference time respectively. Furthermore, in the nine real-world scenarios of four evaluated datasets, TraceCluster outperforms state-of-the-art (SOTA) approaches in terms of overall performance, especially in node-level APT detection tasks.

Index Terms—Provenance graphs, subgraph clustering, attention networks, APT detection.

I. INTRODUCTION

THE field of network security is facing increasingly complex threats, among which advanced persistent threats (APTs) have become one of the most challenging due to their stealthiness and complexity. APTs are characterized by long-term latency, sparse distribution, and hard-to-trace behaviors, which challenge traditional detection methods. Provenance graphs, crucial for APT detection, help track attack paths and identify anomalies by capturing the entire system execution [1]. They parse host events from system logs, covering processes, files, network nodes, and their relationships, while tracing complex dependencies [2] and hidden paths [3] of APTs. Several detection methods [1], [4], [5], [6], [7], [8] show advantages in intrusion detection using provenance graphs. However, the large-scale nature of these graphs, with numerous nodes and complex relationships, results in high computational complexity, coarse-grained detection, and insufficient learning patterns, limiting the practical applicability of existing methods [9].

Existing methods [10], [11], [12] use representative training data and sequence processing techniques (e.g., recurrent neural networks) to model past events and predict future ones, resulting in high time complexity. To address this issue, StreamSpot [13], UNICORN [7] and KAIROS [14] build a flow graph and maintain it in memory to enable faster querying. Additionally, MEGR-APT [15] first partitions the input graph into subgraphs, computes their embeddings, and then constructs an optimized query graph for efficient matching. While effective, these methods are limited to graph-level APT detection, offering insufficient granularity. Consequently, security analysts must perform detailed node-level and edge-level inspections within flagged subgraphs to accurately trace APTs.

Node-level detection methods demonstrate superior capability in addressing coarse-grained detection limitations. Two representative approaches include: (1) representation learning through fixed-size neighborhood sampling [8], and (2) comprehensive graph representation learning based on graph neural networks (GNN) [1], [14], [16]. Sampling-based methods incur information loss, which degrades detection accuracy. In contrast, the whole-graph representation learning approach constructs a node anomaly classifier and utilizes an embedded cyclic database to store node embeddings, significantly

Received 26 April 2025; revised 12 November 2025 and 5 January 2026; accepted 7 January 2026. Date of publication 12 January 2026; date of current version 21 January 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3107300; in part by the National Natural Science Foundation of China under Grant 62172244 and Grant 62572285; in part by Shandong Provincial Natural Science Foundation under Grant ZR2024MF050; in part by Taishan Scholars Program under Grant tsqn202211210; in part by the Key Research and Development Program of Shandong under Grant 2025CXPT082; in part by the “20 New Universities” Project of Jinan City under Grant 202333023 and Grant 202333045; and in part by the Pairing Plan Project of the School of Computer Science and Technology, Qilu University of Technology under Grant 2024JDJH12. The associate editor coordinating the review of this article and approving it for publication was Dr. Fabio De Gaspari. (*Corresponding author: Dawei Zhao.*)

Lijuan Xu, Zicheng Zhao, and Dawei Zhao are with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250353, China, and also with Shandong Provincial Key Laboratory of Industrial Network and Information System Security, Shandong Fundamental Research Center for Computer Science, Jinan 250014, China (e-mail: xulj@sdas.org; zhaozicheng1030@126.com; zhaodw@sdas.org).

Zhen Wang is with the School of Artificial Intelligence, Optics and Electronics (iOPEN) and the School of Mechanical Engineering, Northwestern Polytechnical University, Xi’an, Shaanxi 710072, China (e-mail: zhenwang0@gmail.com).

Chunpeng Ge is with the Joint SDU-NTU Centre for Artificial Intelligence Research (C-FAIR) and the School of Software, Shandong University, Jinan 250014, China (e-mail: gechunpeng2022@126.com).

Digital Object Identifier 10.1109/TIFS.2026.3653175

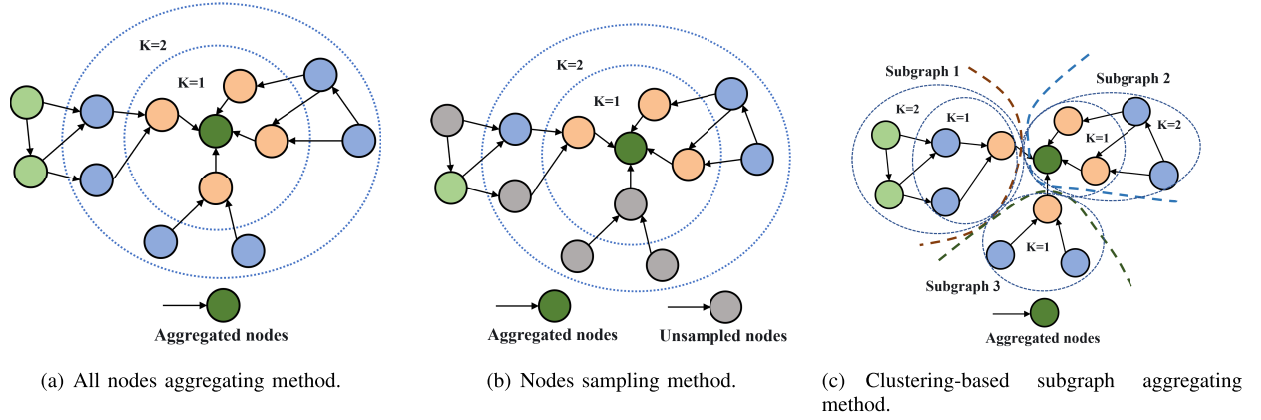


Fig. 1. The comparison of clustering-based subgraph partitioning in TraceCluster, nodes sampling in ThreaTrace, and all node aggregating method.

TABLE I

TIME AND SPACE COMPLEXITY OF GCN TRAINING ALGORITHMS. L REPRESENTS THE NUMBER OF LAYERS, N IS THE NUMBER OF NODES, AND E IS THE NUMBER OF EDGES. F_{in} AND F_{out} ARE THE DIMENSIONS OF INPUT AND OUTPUT FEATURES, RESPECTIVELY. FOR SIMPLICITY, WE ASSUME THAT THE FEATURE DIMENSIONS ARE FIXED ACROSS ALL LAYERS. FOR NEIGHBOR-SAMPLING-BASED APPROACHES SUCH AS GRAPH SAGE AND FASTGCN, S IS THE NUMBER OF SAMPLED NEIGHBORS PER NODE AT EACH LAYER. IN VR-GCN, S CAN BE SMALLER DUE TO THE VARIANCE REDUCTION TECHNIQUE. FOR CLUSTER-GCN AND CLUSTER-SUBGRAPH-ATTENTION, C IS THE NUMBER OF SUBGRAPHS, N_c AND E_c ARE THE AVERAGE NUMBER OF NODES AND EDGES PER SUBGRAPH, RESPECTIVELY. K REPRESENTS THE NUMBER OF ATTENTION HEADS USED IN ATTENTION-BASED METHODS

Method	Time Complexity (Multi-layer)	Space Complexity
GCN [17]	$\mathcal{O}(L(EF_{in} + NF_{in}F_{out}))$	$\mathcal{O}(NF_{in} + LF_{in}F_{out} + E)$
GraphSAGE [18]	$\mathcal{O}(NS^L F_{in}F_{out})$	$\mathcal{O}(NF_{in} + NS^L + LF_{in}F_{out})$
GAT [19]	$\mathcal{O}(LKEF_{out})$	$\mathcal{O}(NF_{in} + LKF_{in}F_{out} + E)$
FastGCN [20]	$\mathcal{O}(LSF_{in}F_{out})$	$\mathcal{O}(SF_{in} + LF_{in}F_{out})$
VR-GCN [21]	$\mathcal{O}(LSF_{in}F_{out} + EF_{in})$	$\mathcal{O}(NF_{in} + SF_{in} + LF_{in}F_{out})$
Cluster-GCN [22]	$\mathcal{O}(CL(E_cF_{in} + N_cF_{in}F_{out}))$	$\mathcal{O}(E_c + N_cF_{in} + LF_{in}F_{out})$
Cluster-Subgraph-Attention	$\mathcal{O}(CLK E_cF_{out})$	$\mathcal{O}(CE_c + N_cF_{in} + LKF_{in}F_{out})$

reducing training time. While ensuring high detection efficiency and fine-grained detection, this method suffers from poor time efficiency during inference. Unlike ThreaTrace and UNICORN, the ORTHRUS [16]'s anomalous node detection approach results in a relatively high false negative rate. That is partly because it no longer classifies the neighbors of anomalous nodes as anomalous, which is a critical concern in cybersecurity analysis.

To address these issues, we propose TraceCluster, a lightweight and adaptive clustering-based APT detection method. TraceCluster employs a clustering-based subgraph partitioning strategy for large-scale provenance graphs, preserving causal dependencies between neighboring nodes while avoiding the high complexity of global graph computation. Additionally, it introduces a novel approach that dynamically captures complex internode relationships within subgraphs, using attention mechanisms to focus on the most relevant connections, significantly reducing computational complexity compared to traditional GCN methods.

Similarly to the typical provenance graph-based APT detection approach [8], TraceCluster learns the role of each benign node within the data provenance graphs to identify stealthy anomalous behaviors without relying on previous knowledge of attack patterns. The training data are benign, while the validation and test datasets contain a large number of benign nodes and a small proportion of anomalous nodes.

We compare our cluster-based subgraph partitioning with all nodes aggregating method and fixed-size neighborhood sampling of ThreaTrace [8] in Figure 1. In order to illustrate the computational efficiency of our proposed approach, we provide Table I, which presents a comparison of time complexity and space complexity between our clustering-based subgraph attention network (SAN) and other models.

Cluster-GCN employs a clustering strategy to improve training efficiency. However, its fundamental design focuses on graph partitioning, where the entire graph is divided into smaller blocks for parallel computation. In particular, it does not incorporate any subgraph-level attention mechanism within these partitions. In contrast, TraceCluster introduces a subgraph-level attention mechanism that dynamically assigns weights to the interactions among nodes within each subgraph. This design enables the model to more accurately capture inter-node dependencies and uncover latent anomalous patterns.

Given the data imbalance between normal and anomalous nodes inherent in APT detection, TraceCluster proposes an adaptive category weighted mechanism in the loss function. This assigns different weights to samples of varying classes, strengthening the model's ability to detect rare anomalies. This is particularly effective in addressing stealthy and covert attacks, ensuring that anomalous samples are emphasized during training.

The contributions of this paper are summarised as follows:

- **Lightweight subgraph partitions and subgraph attention network.** TraceCluster partials the large-scale origin graph into multi-smaller subgraphs and independently performs graph attention operations in each subgraph. Graph partitioning technology not only optimizes computational efficiency, but also preserves the local structural features of the APT behaviour, thereby enhancing the scalability of the model. Furthermore, by aggregation attention based on node adjacency relationships and edge types, TraceCluster generates highly discriminative node feature representations, thereby enhancing node-level threat detection and significantly improving its capability to identify complex APT attacks.
- **Adaptive category weighting mechanism.** We innovatively propose a category weighting mechanism that dynamically assigns weights to different node categories, significantly enhancing the model's ability to detect rare and anomalous behaviours. This approach improves the detection performance for latent and covert attacks while also increasing the model's adaptability in imbalanced data environments.
- **High detection performance and efficiency.** TraceCluster demonstrates excellent detection performance. It achieves 37-fold and 3-fold reductions in inference time compared to the state-of-the-art approaches, respectively. Furthermore, in the nine real-world scenarios of four evaluated datasets, TraceCluster outperforms state-of-the-art approaches (SOTA) in terms of overall performance.

Availability: TraceCluster is available online at <https://github.com/CSNS-Lab/TraceCluster>

The paper is organized as follows: related work is presented in Section II. Section III presents background and motivation. Section IV describes the threat model. In Section V, we present the design of TraceCluster. We show the evaluation performance in Section VI. Section VII is a discussion of limitations. Finally, the full paper is summarised.

II. RELATED WORK

Anomaly-based APT detection methods that regard a system as anomalous by first learning its normal behavioral patterns, and then recognizing it as anomalous when its behavior deviates from these patterns. This type of approach does not rely on predefined attack patterns, and thus is more flexible in dealing with unknown attacks.

DeepLog [23] and Tiresias [24] model normal behaviour and detect abnormal user behaviour through repeated neural networks (RNN), which focus on sequential data, are not sufficiently capable of capturing global relationships in graph structures. Therefore, they are not suitable for large-scale data structures. LogRobust [25] improves the ability to model log sequences through bidirectional long-short-term memory (LSTM) combined with the attention mechanism, which allows it to better capture complex contextual correlation information. However, it has high computational complexity and weak adaptation to unbalanced data in the processing of long-sequence data and complex logs. Instead of directly dealing with the graph structure, LogGAN [26] uses adversarial

training to generate and detect anomalous log events, solving the problem of data imbalance. Although LogGAN performs well in detecting known attacks, it is based on training data and could not effectively identify zero-day attacks. In addition, it also has high resource consumption due to the computational overhead of the adversarial training process of Generative Adversarial Networks (GAN). T-Trace [27] proposes a training-free data-driven APT detection method, which uses tensor decomposition to mine implicit relationships in original logs and pruning techniques to solve the dependency explosion problem. This approach is suitable for dealing with local data structures, however, it lacks the ability to model full graphs or complex graph structures.

Several scholars have endeavored to tackle the issue through learning graph structures. UNICORN [7] extracts the structural features of the provenance graph using the Weissfeiler-Lehman (WL) graph kernel [28] to detect APT attacks through the structural similarity of the graph. However, the graph kernel approach has high computational complexity and low efficiency in large-scale data processing. ThreaTrace [8] detects and tracks threat activities in hosts through provenance graph learning using a node-level feature extraction approach combined with graph structure analysis. Although it uses the GraphSAGE model, which assigns the same importance to all neighbour nodes, its consideration of only a fixed number of neighbours led to sampling issues that caused the loss of important information, reducing the accuracy and sensitivity of anomaly detection.

As evidenced by the aforementioned studies, excessive resource consumption constitutes a critical challenge that cannot be overlooked in provenance graph-based APT detection. ProvG-Searcher [29] shows high efficiency in the search for provenance graphs. The method is based heavily on well-structured input data. This means that in practical applications, if the data structure is unclear, performance may suffer significantly, and the computational cost increases notably when encountering large-scale data. Furthermore, DISTDET [30], while cost-effective in a distributed architecture, faces challenges in the management and coordination of distributed systems, particularly with communication delays and synchronisation issues, which could affect the effectiveness of real-time detection.

SHADEWATCHER [31] formulates provenance-based threat detection as a recommendation task over system entity interactions. It produces fine-grained, interaction-level alerts and supports semi-supervised adaptation with analyst feedback. However, SHADEWATCHER still relies on GPU-heavy training, incurs non-trivial offline overhead for provenance graph construction, leaving room for lighter-weight and more incrementally updatable designs.

NODLink [32] combines VAE-based node anomaly scoring with an online Steiner tree formulation to reconstruct compact attack stories from large provenance graphs. It achieves fine-grained, node-level investigation while maintaining high detection accuracy and practical throughput in realistic SOC environments. However, NODLink uses non-GNN representations, and builds on Steiner Tree Problem (STP)-based optimization together with a graph-database-backed pipeline,

which increases engineering complexity and limits scalability to very large, highly dynamic graphs. These limitations indicate remaining space for more scalable, GNN-based designs that can better capture higher-order structural patterns while retaining fine-grained detection and efficiency, as pursued in this work.

MEGR-APT [15] and FLASH [1] have demonstrated remarkable efficacy in addressing efficiency-related challenges within computational systems. MEGR-APT first partitions the input graph into subgraphs, computes their embeddings, and then constructs an optimized query graph for efficient matching. Despite its efficacy, it is limited to graph-level APT detection, resulting in insufficient granularity. FLASH [1] combines the graph neural network and the provenance graph to improve the detection accuracy through Word2Vec and GNN encoders. It also optimizes computational efficiency using an embedded recycling database, enabling real-time APT detection. Although FLASH maintains high detection efficiency and achieves fine-grained detection, it nevertheless exhibits unsatisfactory time efficiency during the inference phase.

KAIROS [14] applies temporal graph representation learning to model both structural and temporal dependencies in audit logs, and then derives compact attack summary graphs to support analyst investigation, achieving strong detection performance on multiple DARPA TC and OpTC datasets. While effective, KAIROS incurs non-trivial computational overhead for large time windows.

ORTHRUS [16] applies a graph neural network (GNN)-based encoder-decoder to model normal behavior. Detected anomalous nodes are then used to perform attack path reconstruction, generating a compact subgraph that highlights the most relevant entities involved in the attack. Unlike ThreaTrace and UNICORN, the ORTHRUS [16]'s anomalous node detection approach results in a relatively high false negative rate. That is partly because it no longer classifies the neighbors of anomalous nodes as anomalous, which is a critical concern in cybersecurity analysis.

The above analysis of the research landscape reveals that while extensive investigations have been conducted on optimising temporal efficiency, minimising resource expenditure, and improving detection accuracy in large-scale graph processing systems, node-level APT detection mechanisms persistently exhibit three fundamental limitations: unsustainable computational overhead, prohibitive inference latency, and suboptimal operational efficiency metrics.

III. BACKGROUND & MOTIVATION

A. Data Provenance

Data provenance is a directed acyclic graph constructed from audit data representing the relationship between subjects (e.g., processes) and objects (e.g., files) in a system [7], [13]. By capturing detailed contextual relationships among system entities and events, data provenance has been proposed as a promising source for host-based threat detection [1], [8].

B. Motivation Example

We present an example in Figure 2, to illustrate the limitation of state-of-the-art threat detection work and the intuition of our approach. The example is generated from the DARPA OpTC experiment.¹ The dark nodes are those nodes related to the intrusion campaign.

The attacker installs Notepad plus with a malicious upgrade ability in the victim host. The Malicious process “cKfGW.exe” gets “cmd” shell to run commands to find out information about local system, and migrates from process “cKfGW.exe” to process “lsass.exe”, to connect to attacker’s server. To establish persistence via meterpreter, the process “cKfGW.exe” is migrated to process “lwabeat”, who writes a script to C:\Windows\TEMP\myHbYXTpVwX.vbx and set autorun to the registry.

Limitations of state-of-the-art threat detection work: It remains some challenges to detect the threat presented in the example, which leads to limitations of the state-of-the-art threat detection work.

- **Stealthy attack behavior.** Anomaly-based methods, such as Unicorn [7], StreamSpot [13], and ORTHRUS [16], use graph kernel algorithm or GNN to dynamically model the whole graph and detect abnormal graphs by clustering approaches. However, the intrusion may be stealthy, which means that a system’s provenance graph under intrusion campaign may be similar to those of benign systems. In this example, there are hundreds of thousands of benign nodes and less than 200 anomalous nodes. The proportion of anomalous nodes is less than 0.1%, which thus results in a high similarity of the attack graph and benign graph. Therefore, graph-kernel-based methods are insensitive to the small anomalous nodes.
- **Exhibits low traceability.** Graph-level detection methods [15] first partitioning the input graph into subgraphs, and then constructing an optimized query graph for efficient matching. lack the capability of tracing the location of anomalous behavior, which is vital to trace abnormal behavior and fix the system. In this example, these methods can only raise alarms for the whole graph instead of the specific attack entities (e.g., myHbYXTpVwX.vbx).
- **Low time efficiency.** ThreaTrace requires multiple rounds of testing on the model, sometimes up to more than 20 times, to find the optimal test result. Obviously, it suffers from lower inference time. To eliminate this flaw, Flash [1] optimizes computational efficiency using an embedded recycling database. However, querying the database also incurs a certain amount of time overhead. Although MEGR-APT [15] uses a graph query database to improve inference efficiency, it still runs slower than our method. In this example, ThreaTrace, Flash and MEGR-APT require 80 seconds, 32 seconds, and 2.4 seconds, respectively, to perform the inference steps.

¹<https://drive.google.com/drive/u/0/folders/1n3kkS3KR31KUegn42yk3-e6JkZv0Caa>

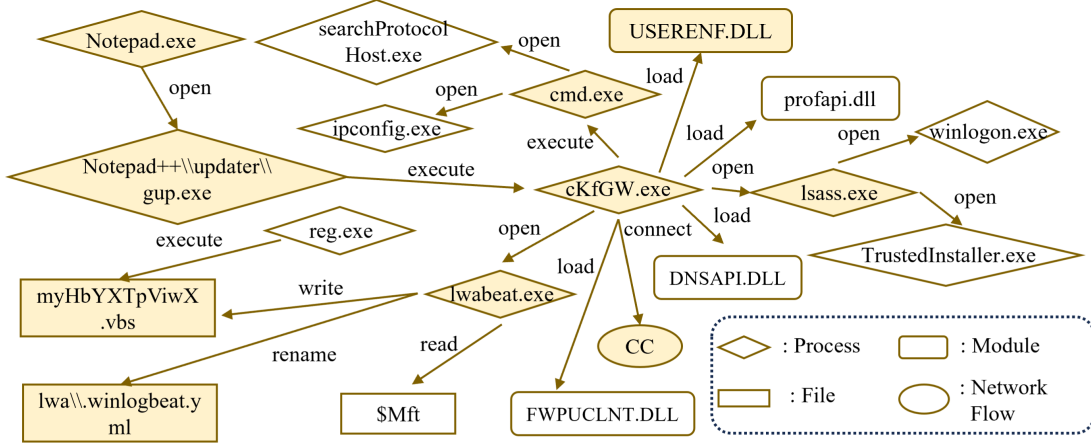


Fig. 2. The example from malicious upgrade scene of DARPA OpTC experiment.

IV. THREAT MODEL

A. Adversary Capabilities

The adversary is characterized as an APT, exhibiting techniques and operational goals that are consistent with those defined in the MITRE ATT&CK knowledge base.²

- **Stealty.** The adversary exploits multiple zero-day vulnerabilities and combined various attack methods to launch a slow infiltration attack on the target system. The attack behavior pattern is similar to normal operations, and the victim system is unable to detect the existence of the attack. Therefore, we are unable to obtain information about its attack pattern.
- **Persistence.** The capability to remain covertly embedded within a compromised system for an extended period, maintaining long-term presence and conducting multiple attacks over time.

In additions, we consider that attack patterns are embedded within the provenance graph. The adversary's actions cause certain nodes in the system's provenance graph to deviate locally from normal behavioral patterns, thereby leaving discernible traces of malicious activity within the provenance data.

B. Assume

In this paper, an entity within a running system is defined as *benign* when it exhibits no association with malicious activities. Conversely, an entity is regarded as *abnormal* if its behavioral patterns deviate from those of benign entities, which commonly results from ongoing intrusion campaigns. Our analysis centers on the nodes of the provenance graph to identify anomalous entities at the host level. We further assume that the system's audit logging and provenance collection mechanisms are both comprehensive and trustworthy, and cannot be tampered with or compromised by adversaries [31].

TraceCluster cannot detect threats which have no different patterns in the provenance graph compared to the benign

entities learned in training phase. We further discuss this limitation in Section VIII.

V. PROPOSED APPROACH

In this section, we present the design of TraceCluster. The main components are shown in Figure 3. We will describe them in further detail in the following subsections.

Algorithm 1 TraceCluster: Subgraph Training and Anomaly Detection

Require: Graph G , node features X , adjacency matrix A , labels Y , number of clusters K , learning rate η , threshold τ

Ensure: Trained Model, List of Abnormal Nodes

```

1:  $clusters \leftarrow \text{METIS.partition}(G, K)$ 
2:  $model \leftarrow \text{SAN}()$ 
3:  $optimizer \leftarrow \text{Optimizer}(model.parameters(), lr = \eta)$ 
4: for  $epoch = 1$  to  $num\_epochs$  do
5:   for each  $cluster$  in  $\text{shuffle}(clusters)$  do
6:      $sub\_nodes \leftarrow cluster.nodes$ 
7:      $sub\_adj \leftarrow A[sub\_nodes, sub\_nodes]$ 
8:      $sub\_features \leftarrow X[sub\_nodes]$ 
9:      $sub\_labels \leftarrow Y[sub\_nodes]$ 
10:     $output \leftarrow model(sub\_features, sub\_adj)$ 
11:     $loss \leftarrow \text{WeightedLoss}(output, sub\_labels)$ 
12:     $optimizer.zero\_grad()$ 
13:     $loss.backward()$ 
14:     $optimizer.step()$ 
15:   end for
16: end for
17:  $abnormal\_nodes \leftarrow []$ 
18: for each  $node, prob$  in  $\text{enumerate}(output)$  do
19:   if  $prob > \tau$  then
20:      $abnormal\_nodes.append(node)$ 
21:   end if
22: end for
23: return  $model, abnormal\_nodes$ 

```

Algorithm 1 describes the implementation of TraceCluster. The input to the model includes the graph G , node features X ,

²<https://www.mitre.org/capabilities/cybersecurity/threat-based-defense>

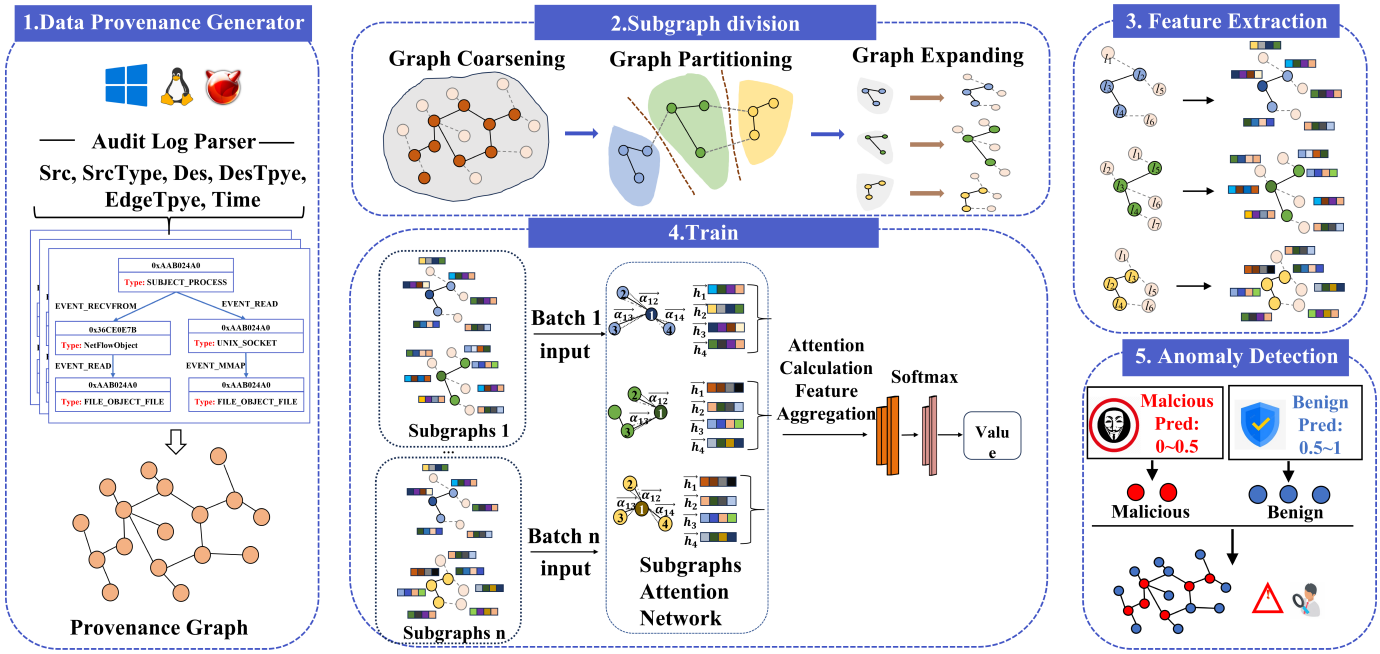


Fig. 3. TraceCluster detection process includes data provenance generator, subgraph division, feature extraction, model train, and final anomaly detection. 1. Extract data from the system log database and external tools to build a provenance graph. 2. Simplify the structure of the global graph by aggregating nodes and edges, and then divide the coarsened global graph into subgraphs. 3. Assign labels to nodes based their types or distributions, and construct the initial feature direction for each node. 4. During the training phase, the subgraphs are then structurally optimized and randomly selected as batches, and attention is used to aggregating the nodes and edges for training the subgraphs in turn to share global graph information. 5. The trained model is used for anomaly detection that combines with the threshold mechanism to classify nodes or edges, and output normal or abnormal status.

adjacency matrix A , labels Y , the number of clusters K , and the threshold τ used for APT detection.

Initially, we adopt the METIS algorithm [33] to partition the graph into K subgraph clusters, stored in the variable clusters. Each cluster is designed to be trained independently. During training, the clusters are randomly selected in each epoch to mitigate bias caused by fixed sequential order. For each cluster, the corresponding subgraph nodes, adjacency matrix, and node features are extracted.

The model performs forward propagation on the subgraph, producing predictions denoted as output.

Once training is complete, the model outputs the probability of anomalies for each node. If the predicted probability for a node exceeds the threshold thr , the node is identified as an anomalous node and added to the list of anomalous nodes. Finally, the trained model and the list of detected anomalous nodes are returned.

A. Provenance Graph Generator

The provenance graph generator illustrates the complete process from system log collection to provenance graph generation. Initially, the system collects log data from various sources, such as servers and endpoint devices. These logs contain information about the subjects (e.g., processes), objects (e.g., files, network streams), and the interaction events between them. All collected logs are stored in the System Log Database (System Log DB), providing a stable foundation for subsequent processing.

Next, external tools such as Camflow [34] are utilized to analyse and process stored log data. Camflow captures

and constructs information flows in chronological order while ensuring robust data integrity and security. During this process, the relationships between subjects, objects, and their associated events are clearly extracted, forming a structured data model.

Finally, the processed data are transformed into a provenance graph, which visually represents the entities in the system and their interrelations. In a provenance graph, the nodes represent the subjects and objects, such as files, network streams, or system processes, while the edges illustrate the interactions between these entities. This visual representation enables analysts to quickly identify system behaviors, dependencies, or anomalies, effectively trace attack paths, analyse threat origins, and improve the accuracy and efficiency of intrusion detection and provenance analysis.

B. Subgraph Division for Large-Scale Graphs

Subgraph division plays a critical role in reducing computational complexity and improving computational efficiency when dealing with large-scale graph tasks. We adopt a multi-stage strategy to effectively partite large graphs while ensuring high partition quality. The process of subgraph division mainly includes three main steps: graph coarsening, initial partitioning, and graph expanding.

1) *Graph Coarsening*: During the initial step of subgraph division, graph coarsening is performed to reduce the graph size. The input graph $G = (V, E)$ is transformed by merging nodes and creating coarse-level representations. Specifically, given two nodes u and v , if there exists an edge (u, v) in the graph, the two nodes can be merged into a single supernode.

The resulting coarsened graph is denoted as $G' = (V', E')$, where V' represents the set of supernodes, and $|V'| \ll |V|$.

Through this approach, the size of the graph is significantly reduced, facilitating the subsequent partitioning process. The coarsening process is typically implemented using maximum matching algorithms, where nodes with relatively higher edge weights are prioritized for merging. The goal of coarsening is to preserve the overall structure and connectivity of the graph while minimizing the number of nodes and edges, thus laying the foundation for the initial partitioning phase.

2) *Graph Partitioning*: After coarsening the graph, the next step is to perform the initial partitioning of the coarsened graph G' . The goal of partitioning is to divide the graph into subgraphs k , $S_1, S_2, \dots, S_k$, while achieving two main objectives:

- **Minimize partition size.** This objective aims to reduce the number of edges between different subgraphs as much as possible.
- **Balancing the load of the nodes.** During the partitioning process, the nodes of each subgraph are kept relatively balanced to ensure load balance for subsequent computations.

In large-scale graph applications, such as anomaly detection, minimizing the partition size is essential to improve the detection accuracy. Reducing the edge between subgraphs helps reduce communication overhead and data transmission bottlenecks, thereby enhancing computational efficiency during further refinement steps. This is particularly critical for complex graphs, where minimizing partition size directly alleviates computational burdens.

The partition size can be mathematically defined as:

$$\text{partition}(S_1, S_2, \dots, S_k) = \sum_{i=1}^k \sum_{u \in S_i, v \in S_j} w(u, v), \quad (1)$$

where $w(u, v)$ represents the weight of the edge between nodes u and v .

Maintaining load balance in graph partitioning is essential for improving parallel computational performance and model scalability. A balanced partition ensures that the computational resources allocated to each subgraph are roughly equal, preventing the computational load imbalance caused by uneven subgraph sizes. The load imbalance can impact the training process, as different subgraphs may contain varying distributions of nodes and edges, which could result in inconsistent learning outcomes. By balancing the partitioning process, the model ensures stability during training and avoids problems caused by unequal learning rates or data updates due to size discrepancies among subgraphs.

The condition for load balance can be expressed as:

$$|S_i| \approx \frac{|V|}{k}, \quad \forall i \in \{1, 2, \dots, k\}, \quad (2)$$

where S_i represents the set of nodes in subgraph i , $|V|$ is the total number of nodes, and k is the number of subgraphs.

By simultaneously minimising the cut size and maintaining the load balance, we can achieve high computational efficiency while improving the accuracy and stability of the model.

3) *Graph Expanding*: After completing the initial partitioning of the coarsened graph, the expansion process begins, where the coarsened graph G' is progressively restored to its original size G . In this step, each supernode is decomposed back into its original nodes, and these nodes inherit the subgraph assignments from the coarsened graph.

The partition results are propagated incrementally from the coarsened graph to the original graph, ensuring that the assignments maintain consistency. At each stage, the partitioning results are refined to achieve better nodes and edges balance while reducing edge cuts. The process recovers the original modular structure of the graph, and the partition quality achieved during the initial partitioning stage is further enhanced.

Local optimisation techniques are applied to further adjust and optimize partition results. The objective of these techniques are to reduce edge partitioning by selectively moving nodes between subgraphs while maintaining balance among the subgraphs.

Refinement typically uses algorithms such as Kernighan-Lin [35] or Fiduccia-Mattheyses methods [36]. These algorithms iteratively optimize the placement of the nodes by evaluating and exchanging nodes across subgraph boundaries to minimize the cut size. Nodes that can be moved to a neighboring subgraph without violating the load balance condition are identified. If such a move leads to a significant reduction in edge cuts, the node is reassigned.

C. Feature Extraction

Detecting anomalous nodes in the absence of prior knowledge is a significant challenge. Traditional methods rely on annotated data for training. However, such methods are ineffective when faced with novel attack modes. Previous approaches assumed that neighboring nodes are highly similar, but this assumption does not always hold in heterogeneous graphs (e.g., nodes with varying types and connections). Therefore, it is necessary to adopt a more adaptive feature extraction approach.

To address this, TraceCluster combines initial label assignment with node feature initialisation, enabling efficient anomaly detection under scenarios without prior knowledge.

1) *Label Assignment*: Initially, nodes are assigned labels based on their types or distributions. These labels serve as auxiliary information for model learning, helping to distinguish nodes of different types. Assume that each node v_i has a specific type label, such as process nodes, file nodes, or network connection nodes. For each node v_i , the label l_i is defined as:

$$l_i = \text{type}(v_i). \quad (3)$$

2) *Feature Initialisation*: Based on the connection relationships between the nodes, the distribution of the edge types is used to construct the initial feature direction for each node. Suppose that the input graph is $G = (V, E)$, where V represents the set of nodes and E represents the set of edges. For each node $v_i \in V$, its feature vector $\mathbf{a}_i$ consists of two parts: *incoming* feature information and *outgoing* feature information.

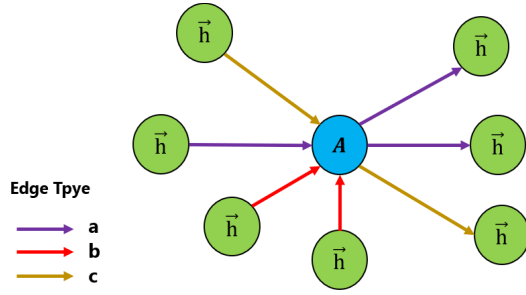


Fig. 4. Examples of feature extraction.

Assume that there are t edge types. For node v_i , the dimension of its feature vector $\mathbf{a}_i$ is $2t$: the first dimensions t represent the incoming edge information and the remaining dimensions t represent the outgoing edge information. The feature vector for node v_i is expressed as:

$$\mathbf{a}_i = [d_{in}^{(1)}, d_{in}^{(2)}, \dots, d_{in}^{(t)}, d_{out}^{(1)}, d_{out}^{(2)}, \dots, d_{out}^{(t)}], \quad (4)$$

where $d_{in}^{(t)}$ and $d_{out}^{(t)}$ represent the number of incoming and outgoing edges of type t for node v_i .

As illustrated in Figure 4, if the feature vector of a node is $\mathbf{a}_i = [1, 1, 2, 2, 1, 0]$, it indicates that the node has varying numbers of incoming and outgoing edges of different types. Specifically, the first three values $[1, 1, 2]$ represent the counts of incoming edges of three types of edges, while the last three values $[2, 1, 0]$ represent the counts of outgoing edges. By combining label assignment with feature initialisation, our unsupervised framework enables node-level anomaly detection without the need for prior anomaly data. This approach also provides robust representations of foundational features, laying the solid foundation for subsequent clustering or classification tasks.

D. Model Training and Optimization

In terms of model training and optimization, TraceCluster integrates adaptive characteristics with efficient graph partitioning methods to maximize the detection capability of anomalous nodes in large-scale heterogeneous graphs. The training and optimization process focuses on the following aspects.

1) *Dynamic Subgraph Training Strategy*: TraceCluster first partitions the heterogeneous graph into multiple subgraphs. Each subgraph is treated as an independent training sample.

To ensure complete learning of node features in large-scale heterogeneous graphs, TraceCluster adopts a mini-batch-like dynamic subgraph training strategy. In each training iteration, one or more subgraphs are randomly selected for training and the subgraphs are periodically replaced during the training process. This ensures that the model learns comprehensive information from different subgraphs. During each iteration, the nodes obtained from the dynamically sampled subgraphs are used to calculate the batch loss function $\mathcal{L}_{batch}$.

The batch loss function is defined as:

$$\mathcal{L}_{batch} = - \sum_{i \in \mathcal{I}} w_{y_i} \log(p(y_i | \mathbf{x}_i)), \quad (5)$$

where $\mathcal{I}$ is the set of nodes sampled from the mini-batch subgraph, y_i is the true label of node i , $p(y_i | \mathbf{x}_i)$ is the predicted probability of node i belonging to class y_i , and w_{y_i} is the weight assigned to class y_i . Specifically, w_{y_i} based on class sample counts, is used to address the class imbalance problem. We describe w_{y_i} in V-D2.

By applying attention operations to a set of subgraphs, SAN dynamically adjusts the importance coefficients between the neighboring nodes and the central node. The attention mechanism computes the importance coefficient α_{ij} of a neighboring node j to a central node i as follows:

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{d}^T [\vec{h}_i \| \vec{h}_j]\right)\right)}{\sum_{k \in N(i)} \exp\left(\text{LeakyReLU}\left(\vec{d}^T [\vec{h}_i \| \vec{h}_k]\right)\right)}, \quad (6)$$

where $\vec{h}_i$ and $\vec{h}_j$ represent the feature vectors of nodes i and j , respectively, and $N(i)$ denotes the set of neighbors for node i . The learnable vector $\vec{d}$ is employed to measure the correlation between nodes. This vector is initialized using the Xavier uniform distribution at the beginning of training, and is subsequently updated together with other model parameters through backpropagation and an optimizer. In this way, $\vec{d}$ gradually learns a weight direction that reflects the relative importance of neighboring nodes.

Through this strategy, SAN not only improves anomaly detection performance within local subgraphs, but also improves the overall structural understanding of the heterogeneous graph. This enables TraceCluster to effectively identify the multistage attack phases and intricate behaviors of APT.

2) *Class Weight Design*: Specifically, classes with fewer samples are assigned higher weights, while classes with larger sample counts are assigned lower weights. The class weight w_k is inversely proportional to the sample count for class k , which is calculated as follows:

$$C_k = \sum_{i=1}^N \mathbb{1}(y_i = k) \quad (7)$$

$$w_k = \frac{1}{C_k}, \quad (8)$$

where N is the total number of nodes, and $\mathbb{1}(y_i = k)$ is an indicator function that evaluates to 1 when node i belongs to class k , and 0 otherwise. To prevent the class weights from being excessively large or small, we normalize the weights so that their sum equals 1. The normalized weight w'_k is defined as:

$$w'_k = \frac{w_k}{\sum_{k=1}^K w_k}. \quad (9)$$

The calculated class weights w'_k are then applied to the batch loss function during the training process. Specifically, for a node, the loss value is weighted by its corresponding class weight, as shown in Equation 5. Through this method, SAN focuses more on classes with fewer samples during the training process, effectively mitigating the impact of class imbalance. This not only improves the predictive accuracy for under-represented classes, but also enhances the overall performance of the classification task.

E. Anomaly Detection

After the model training is completed, the final output is the anomaly probability for each node. The model categorizes the nodes and generates a probability distribution indicating the nodes. During the testing phase, the model leverages these probabilities to determine if a node is potentially anomalous.

If the probability of an anomaly for a node exceeds a predefined threshold, the node is marked as anomalous, indicating that it might be subject to an APT attack. Similar to most expert-recognized approaches [1], [8], TraceCluster learns baseline patterns of normal behavior from benign data, thus eliminating the need to incorporate labels indicating whether a node is malicious during the training phase. In the anomaly detection phase, this work first predicts the probability distribution of node labels. If the classification probability exceeds a predefined threshold of 0.5, the node is considered potentially malicious. Following the methodology in [1] and [8], the final hyperparameter values are determined by exhaustively searching the hyperparameter space.

This probability-based anomaly detection mechanism allows TraceCluster to effectively identify hidden attack paths and anomalous nodes in complex APT attack scenarios. By outputting anomaly probabilities, security analysts can further assess potential threats, analyze their severity, and provide actionable insights to detect and mitigate APT attacks.

VI. EXPERIMENTATION AND ANALYSIS

In this section, we evaluate the detection and inference time performance of the proposed TraceCluster.

Hardware Configuration: Our experiments are performed on a workstation running Ubuntu 22.04.3 LTS, equipped with an AMD Ryzen Threadripper PRO 9955WX (16 cores), an NVIDIA RTX 5090, 64GB memory, and a 1TB SSD.

Preprocessing Costs: TraceCluster only parses the raw logs and extracts the required fields (e.g., entity identifiers, event types, timestamps) without any expensive feature engineering or additional model-dependent computation. Therefore, the preprocessing overhead is negligible compared to the subsequent graph-based inference pipeline.

A. Baseline Models and Datasets

1) *Baseline Models:* Ten algorithms are chosen as baselines, that is, StreamSpot [13], UNICORN [7], Log2Vec [11], DeepLog [23], LogRobust [12], LogGAN [26], ThreaTrace [8], FLASH [1], KAIROS [14], and ORTHRUS [16], whose details have been described in Section II. We do not include MEGR-APT [15] as a baseline for detection performance comparison since it is a graph-level detection method. However, due to its relatively fast detection speed, we adopt it as the baseline model for inference time evaluation.

2) *StreamSpot:* StreamSpot³ (Table II) is an experimental dataset designed to study network anomaly detection [13]. It includes six scenarios: five representing normal behaviors and one representing an attack behavior. The dataset is stored in tab-delimited file format, where each row represents an edge,

TABLE II
OVERVIEW OF STREAMSPOT DATASET

Scene	# of graph	Average # of nodes	Average # of edges
Benign	500	8,315	173,857
Attack	100	8,891	28,423

TABLE III
OVERVIEW OF DARPA TC # E3 AND # E5 DATASETS

Dataset	Scene	#benign nodes	# abnormal nodes	# edges
# E3	Trace	1,144,065	67,383	6,978,024
	Cadets	344,322	12,852	8,663,569
	Fivedirections	374,059	425	9,852,465
# E5	Theia	31,762	19,365	126,824,068
	Trace	261,078	124,003	3,165,213
	Cadets	283,754	7,563	11,248,647

containing the source node ID, source node type, destination node ID, destination node type, edge type, and graph ID. It contains 6*100 information flow graphs derived from five benign scenes and one attack scene. Each scene runs 100 times to generate 100 graphs using the Linux System Tap logging system [37]. We use the same validation strategy as [8] to randomly split the dataset into a training set with 75*5 benign graphs and a testing set with 25*5 benign graphs and 25 attack graphs.

3) *DARPA TC # E3:* DARPA TC project's Engagement 3 dataset⁴ (Table III) was produced as part of DARPA's Transparent Computing initiative [38]. The third engagement specifically targets the exploration and analysis of APT behaviors at a foundational level. The # E3 dataset encompasses multiple attack scenarios, including post-compromise activities, spear-phishing attacks, data exfiltration, APT attacks, etc. The dataset provides fine-grained data, including system processes, files, network activities, and system calls, making it suitable for basic anomaly detection and preliminary threat source analysis.

The # E3 dataset lasted for two weeks. During the engagements, the red team performed attacks and benign background activity to hosts with different systems. Three kinds of attackers are involved in the red-team including Nation State, Common Threat and Metasploit. The provenance collecting tools captured provenance data of the whole system from start to end. The provenance data and ground truth are publicly available [38]. Therefore, we use the ground truth to label the abnormal nodes and evaluate TraceCluster's performance of detecting them, which shows the ability of tracking anomaly. The Details of validation strategy is similar to [8]. Files that have no threats are used to train the models and files with threats are used for evaluation.

4) *DARPA TC # E5:* Building upon the foundation of the # E3 dataset, the # E5 dataset⁵ (Table III) lasted for 9 days, is larger in scale and designed with more sophisticated

⁴<https://github.com/darpa-i2o/Transparent-Computing/blob/master/README-E3.md>

⁵<https://github.com/darpa-i2o/Transparent-Computing>

³<https://github.com/sbstreamspot/sbstreamspot-data>

TABLE IV
OVERVIEW OF DARPA OPTC DATASET

Scene	# benign nodes	# abnormal nodes	# edges
Powershell Empire	553,130	410	582,194
Malicious Upgrade	181,724	178	169,555

attack scenarios, including multistage attack chains, multi-host collaboration, data exfiltration and encryption, antiforensic activities, and advanced network communications. It is especially valuable for analysing the behavioural patterns and defensive strategies associated with complex APT attacks.

We use the same ground truth and validation strategy with the # E3 dataset.

5) *DARPA OpTC*: The DAPRA OpTC dataset⁶ (Table IV) is an open source resource designed for network security research and attack detection, containing audit logs from approximately 1,000 hosts. The dataset includes 7 days of benign activity data and 3 days of malicious activity data, with each day of malicious activity representing a different type of attack. We use benign activity data for training, and utilize labels of malicious activity from FLASH [1]. For evaluation purposes, we focus on the attack data from Day 2 and Day 3. On day 2, the logs document an attacker who sends phishing emails with a malicious “Custom PowerShell Empire stager”. The stager is used to escalate privileges, exfiltrate data, and perform lateral movement on the domain controller. The attacker establishes persistence through WMI subscriptions and transfers data using RDP and SSH. On day 3, the dataset records an attacker exploiting a “malicious upgrade” vulnerability in the Notepad Plus software. The attacker remotely downloads and executes a malicious program called `update.exe`, gaining access to the system, performing lateral movement, using Meterpreter to collect sensitive information and establishing persistence.

For data partitioning, we split all benign data into training and validation sets with an 8:2 ratio, and use the datasets containing malicious nodes exclusively as the test set.

B. Comparison of Detection Performance Between TraceCluster and Baseline Advanced Detection Models

1) *Comparison and Analysis of Detection Performance on StreamSpot Dataset*: In this section, we discuss the performance primarily in the StreamSpot dataset, as shown in Table V. We adopt the same validation strategy as UNICOIN [7], randomly splitting the dataset into a training set containing 75*5 benign graphs and a test set containing 25*5 benign graphs and 25 attack graphs. The TP/FP/TN/FN values in the table represent the average evaluation results obtained by repeating this process. The TraceCluster exhibits exceptional overall capability in detection tasks, effectively maintaining a low false positive rate while achieving near-perfect positive coverage (extremely low false negative rate).

Compared to other models, TraceCluster shows significant performance advantages. Even when StreamSpot and

UNICORN perform well, TraceCluster surpasses both models across all metrics, particularly with a Recall of 0.99, indicating that the model can detect nearly all positive cases.

2) *Comparison and Analysis of Detection Performance on DARPA TC # E3 and # E5 Datasets*: As shown in Table VI, due to the large number of DARPA TC datasets, we chose to test the dataset used on the FLASH [1], and the experimental results are shown in the table. The rest of the results were compared based on the results in ThreaTrace [8]. We run KAIROS [14] and ORTHRUS [16] on # E3 Cadets, # E5 Theia, and # E5 Cadets datasets, as their approaches exhibit relatively high efficiency on the corresponding datasets reported in their original papers. Experimental results indicate that TraceCluster achieves the best performance in terms of F1-score and False Positive Rate (FPR) across the six evaluation scenarios. Although its recall is slightly lower, the precision remains satisfactory. The False Negative Rate (FNR) is not the lowest, however, except for the results on the E5 Cadets dataset, it stays within an acceptable range. On the # E5 Cadets dataset, all competing methods, except KAIROS, exhibit unsatisfactory FNR performance. The performance degradation on the # E5 Cadets dataset can be attributed to its higher diversity and noise level, which make it more challenging for provenance-graph-based approaches to capture consistent behavioral patterns.

In the # E5 Theia scenario, TraceCluster achieves a Precision of 0.97 and a Recall of 0.94. These results indicate that the model effectively identifies positive samples while maintaining controlled false positives. In the Trace scenario, TraceCluster achieved nearly perfect results, with all key metrics reaching 0.99, highlighting the model’s exceptional ability to distinguish between positive and negative samples while keeping the false negative rate extremely low.

The superior performance of TraceCluster can be attributed to the following factors. Firstly, Through the attention mechanism, SAN assigns varying weights to neighboring nodes, allowing the model to focus more effectively on critical nodes. Secondly, the introduction of class weight balancing plays a positive role in addressing imbalanced datasets. In scenarios where class imbalance exists, the weighting mechanism enhances the model’s ability to identify minority classes, reducing the problem of minority samples being overlooked, and thereby improving Recall.

In terms of FPR and FNR, TraceCluster demonstrates high accuracy in distinguishing between normal and attack samples. In # E3 Trace and # E3 Fivedirections datasets, its FPRs are 0.01 and 0.0001, respectively, which are significantly lower than those of other models, indicating its higher stability and precision. The model effectively reduces false positives and improves the reliability of attack detection. Additionally, its sensitivity in detecting attack samples is also remarkable, with FNRs of 0.01 in the # E5 Trace datasets, which are much lower than those of other models, highlighting its advantages in increasing recall and reducing false negatives.

However, TraceCluster also has certain limitations. In the # E5 Cadets scenario, for instance, TraceCluster exhibits a relatively low Recall of 0.80, compared to the best-performing detection model (e.g., KAIROS) at 1.00. This suggests that

⁶<https://drive.google.com/drive/u/0/folders/1n3kkS3KR31KUegn42yk3-e6JkZvf0Caa>

TABLE V
COMPARISON OF PERFORMANCE METRICS ON STREAMSPOT DATASET

Dataset	Model	Precision	Recall	F1-Score	FPR	FNR	TP	TN	FP	FN
StreamSpot	StreamSpot [13]	0.73	0.91	0.81	0.066	0.088	22.8	116.7	8.3	2.2
	UNICORN [7]	0.95	0.97	0.96	0.011	0.027	24.32	123.64	1.36	0.68
	TraceCluster	0.96	0.99	0.98	0.002	0.008	24.8	124.8	0.2	0.2

TABLE VI

EVALUATED UNDER SIX DIFFERENT ATTACK SCENARIOS OF DARPA TC # E3 AND # E5. ALL VALUES ARE REPORTED IN PERCENTAGES, AND THE BEST RESULTS ARE HIGHLIGHTED IN BOLD. FPR: FALSE POSITIVE RATE, REFERRING TO THE PROPORTION OF NORMAL SAMPLES INCORRECTLY CLASSIFIED AS ATTACK SAMPLES; FNR: FALSE NEGATIVE RATE, REFERRING TO THE PROPORTION OF ATTACK SAMPLES INCORRECTLY CLASSIFIED AS NORMAL SAMPLES

Dataset	Detector	Precision	Recall	F1-score	FPR	FNR	TP	TN	FP	FN
# E3 Trace	Log2vec [11]	0.54	0.78	0.64	0.039	0.2189	52,631	1,098,826	45,239	14,752
	LogRobust [12]	0.51	0.75	0.61	0.043	0.25	50,537	1,094,729	49,336	16,846
	ThreaTrace [8]	0.73	0.99	0.84	0.021	2.9×10^{-5}	67,381	1,119,186	24,879	2
	FLASH [1]	0.85	0.99	0.91	0.001	0.01	66,709	1,132,293	11,772	674
	TraceCluster	0.98	0.95	0.96	0.001	0.05	64,014	1,142,693	1,307	3,369
# E3 Fivedirections	Log2vec [11]	0.22	0.76	0.34	0.003	0.24	323	372,917	1,142	102
	LogRobust [12]	0.12	0.64	0.21	0.005	0.36	272	372,064	1,995	153
	ThreaTrace [8]	0.74	0.86	0.79	0.0003	0.1317	369	373,930	129	56
	FLASH [1]	0.72	0.93	0.80	0.0004	0.0706	395	373,905	154	30
	TraceCluster	0.94	0.90	0.92	0.0001	0.0988	383	374,035	24	42
# E3 Cadets	Log2vec [11]	0.49	0.85	0.62	0.033	0.1483	10,945	332,946	11,376	1,907
	LogRobust [12]	0.41	0.83	0.55	0.044	0.169	10,679	328,964	15,358	2,173
	ThreaTrace [8]	0.73	0.99	0.85	0.013	7.8×10^{-5}	12,851	339,802	4,520	1
	FLASH [1]	0.90	0.99	0.94	0.004	0.01	12,724	342,908	1,414	129
	KAIROS [14]	0.80	1.00	0.88	0.010	0.0	13,852	340,859	3,463	0
	ORTHRUS [16]	0.52	0.36	0.43	0.012	0.6398	4,627	340,051	4,271	8,225
	TraceCluster	0.97	0.94	0.95	0.0011	0.06	12,081	343,948	374	771
# E5 Theia	Log2vec [11]	0.57	0.88	0.69	0.403	0.12	17,041	18,938	12,824	2,324
	LogRobust [12]	0.56	0.88	0.68	0.421	0.12	17,041	18,396	13,366	2,324
	ThreaTrace [8]	0.70	0.92	0.80	0.240	0.08	17,816	24,126	7,636	1,549
	KAIROS [14]	0.66	1.00	0.79	0.314	0.0	19,365	21,786	9,976	0
	ORTHRUS [16]	0.85	0.99	0.91	0.110	0.01	19,171	28,258	3,504	194
	TraceCluster	0.97	0.94	0.95	0.019	0.01	19,171	31,169	593	194
# E5 Cadets	Log2vec [11]	0.48	0.73	0.58	0.016	0.2715	5,510	279,192	4,562	2,053
	LogRobust [12]	0.47	0.73	0.57	0.022	0.2715	5,510	277,537	6,217	2,053
	ThreaTrace [8]	0.63	0.86	0.73	0.013	0.14	6,504	279,935	3,819	1,059
	KAIROS [14]	0.43	1.00	0.60	0.035	0.0	7,563	273,729	10,025	0
	ORTHRUS [16]	0.86	0.18	0.31	0.018	0.82	1,362	283,531	223	6,201
	TraceCluster	0.84	0.80	0.82	0.004	0.2	6,050	282,604	1,150	1,513
# E5 Trace	Log2vec [11]	0.50	0.58	0.53	0.275	0.4195	71,982	189,096	71,982	52,021
	LogRobust [12]	0.40	0.47	0.43	0.333	0.5308	58,188	173,796	87,282	65,815
	ThreaTrace [8]	0.81	0.85	0.83	0.095	0.15	105,403	236,368	24,710	18,600
	TraceCluster	0.99	0.99	0.99	0.004	0.010	122,764	260,978	1,100	1,239

TraceCluster may miss some positive samples, failing to capture all correct instances. This shortcoming might arise from the SAN, which is heavily based on node-specific feature representations.

3) *Comparison and Analysis of Detection Performance on DARPA OpTC Dataset:* In the OpTC dataset, we compare TraceCluster with FLASH based on the reproduction results achieved using the official FLASH implementation. The experimental results are shown in Table VII. TraceCluster exhibits a more balanced detection capability in both attack scenarios, with significantly higher Recall, F1-scores and FNR than FLASH. Compared with its relatively lower FNR, the TraceCluster exhibits a slightly higher FPR than FLASH.

Nevertheless, in two evaluation scenarios, the FPR are merely 0.007% and 0.008%, which are practically negligible. Hence, the overall detection performance demonstrates a superior trade-off between accuracy and efficiency. This indicates that TraceCluster is more reliable in detecting a wider range of malicious activities, especially in large-scale or covert attacks.

C. Comparison of Time Consumption for Inferring on DARPA OpTC Dataset

Figure 5 presents a comparison of inference latency between TraceCluster and FLASH under two representative attack scenarios in DARPA OpTC Dataset. TraceCluster enables parallel inference on partitioned subgraphs, significantly reducing the

TABLE VII
COMPARISON OF PERFORMANCE METRICS BETWEEN TRACECLUSTER AND FLASH ON THE DARPA OPTC DATASET

Dataset	Custom Powershell Empire								
Metric	Precision	Recall	F1-score	FPR	FNR	TP	TN	FP	FN
FLASH [1]	0.96	0.52	0.67	1.68×10^{-5}	0.48	213	553,121	9	197
TraceCluster	0.89	0.84	0.86	7.77×10^{-5}	0.161	344	553,087	43	66

Dataset	Malicious Upgrade								
Metric	Precision	Recall	F1-score	FPR	FNR	TP	TN	FP	FN
FLASH [1]	0.92	0.78	0.84	6.6×10^{-5}	0.219	139	181,712	12	39
TraceCluster	0.92	0.87	0.90	7.1×10^{-5}	0.129	155	181,711	13	23

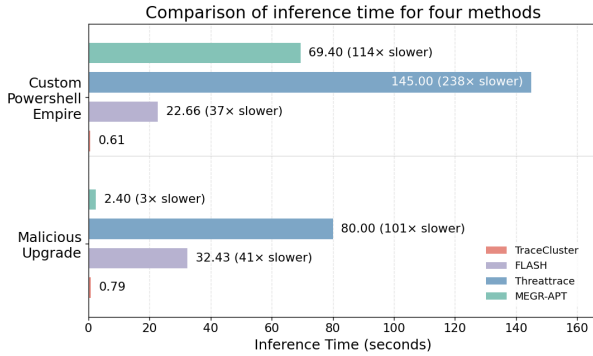


Fig. 5. Comparison of inference time on OpTC dataset.

inference time. Specifically, it achieves inference times of 0.61 s and 0.79 s in the two scenarios, which are substantially lower than FLASH’s 22.66 s and MEGR-APT’s 2.4 s, representing approximately 37-fold and 3-fold, respectively.

We further analyze the reason behind FLASH’s slower inference. The main factor lies in its use of a heterogeneous two-stage inference pipeline. During deployment, FLASH does not perform end-to-end inference via a GNN; instead, it relies on node embeddings pregenerated by a trained GNN model, which are stored in a key-value database. At runtime, the system retrieves these embeddings, combines them with Word2Vec representations of the input events to form feature vectors, and feeds them into an XGBoost classifier for anomaly detection. This process introduces additional delays due to embedding retrieval, feature composition, and external model invocation, resulting in higher overall latency. In contrast, our TraceCluster offers a streamlined inference path and is therefore more suitable for timely detection tasks.

D. Impact of Subgraph Partitioning on TraceCluster

Figure 6 shows the experimental results with different numbers of clusters on the DARPA TC # E3 and # E5 datasets. The results indicate that subgraph partitioning significantly improves model performance, although the sensitivity to the number of partitions varies between scenarios. As can be observed, TraceCluster exhibits high sensitivity to the number of clusters on the # E3 and # E5 datasets. When the number of clusters is less than 6, the F1-score Mean, Precision Mean, and Recall Mean all increase with the growing number of clusters. In a few scenarios (e.g., # E3 Trace and # E5 Cadets),

these metrics show an upward trend when the number of clusters reaches 8. However, for most other scenarios, when the number of clusters exceeds 6, the metrics either decline or remain roughly the same as those recorded when the number of clusters is 6. The experimental results indicate that setting the number of clusters K to 6 yields the best overall performance.

Partitioning is executed once per constructed traceability graph snapshot and can be cached and reused if multiple inference runs or queries are performed on the same snapshot, thereby allowing its cost to be amortized in practical deployment settings.

In conclusion, a well-designed subgraph partitioning strategy helps models better capture the structural information of the data, significantly enhancing prediction accuracy and generalization capability, thus providing deeper insights for tasks in complex scenarios.

E. Selection of Learning Rate for Model Training

As shown in Figure. 8, in this experiment, we select a learning rate of 10^{-2} . This learning rate is chosen based on multiple experiments and comparative analysis, as it effectively balances model stability and convergence speed. Specifically, while smaller learning rates (e.g., 10^{-3} or lower) are stable, they lead to slower convergence, and larger learning rates (e.g., 10^{-1} or higher) can cause training instability or oscillations. Through preliminary experiments, we consider that a learning rate of 10^{-2} avoids excessive oscillations and ensures performance improvement in the # E3 scenarios, which is why it is selected as the final learning rate setting.

F. Selection of Threshold

We refer to the threshold selection strategies described in ThreaTrace [8], KAIROS [14] and FLASH [1]. All three works adopt a fixed-threshold approach. Therefore, we follow the same strategy in our implementation. In addition, we employ a search strategy to determine the optimal values of three hyperparameters, namely the learning rate, the number of clusters, and the threshold. We present the results of the threshold selection experiments under the optimal learning rate 10^{-2} and clustering settings ($K=6$). From Figure. 7, we observe that setting the threshold to 0.5 yields a good tradeoff among Precision, Recall, and F1-score for TraceCluster on the # E3 dataset.

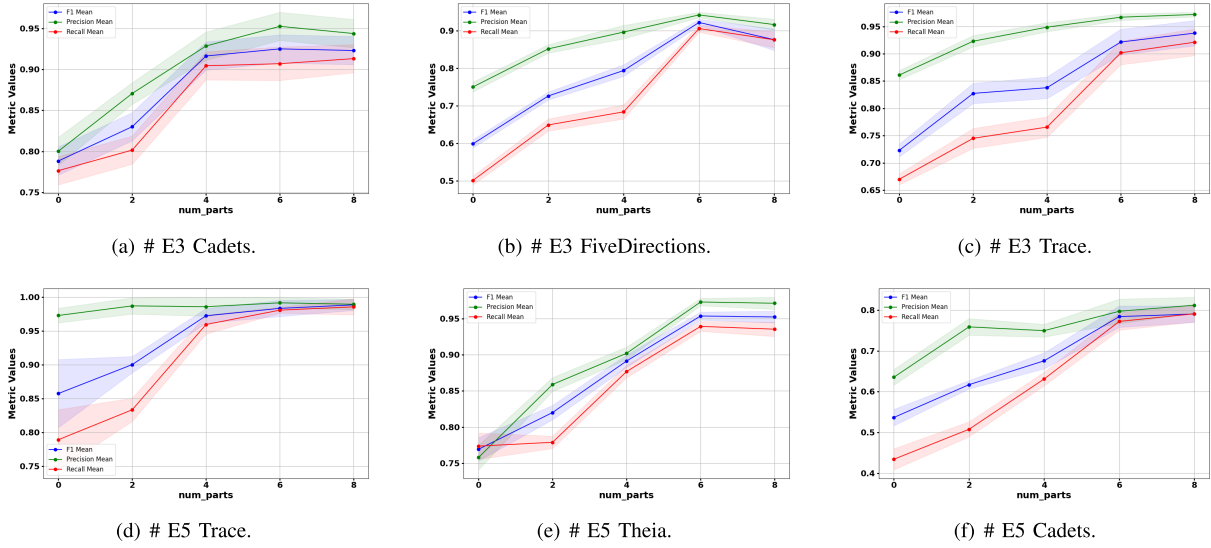


Fig. 6. Experimental results with different numbers of clusters on DARPA TC # E3 dataset and # E5 dataset.

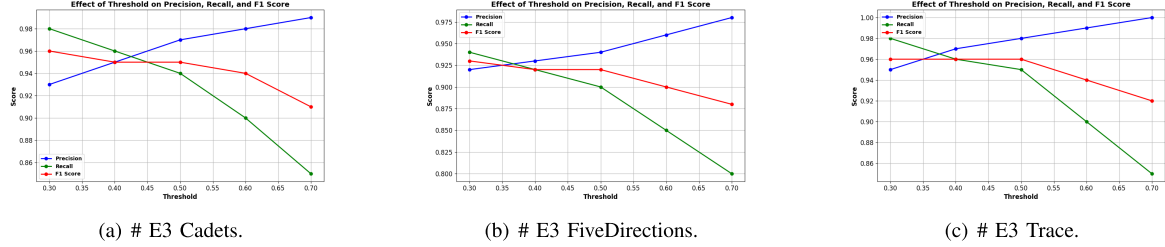


Fig. 7. Experimental results with different scene thresholds on the DARPA TC# E3 dataset.

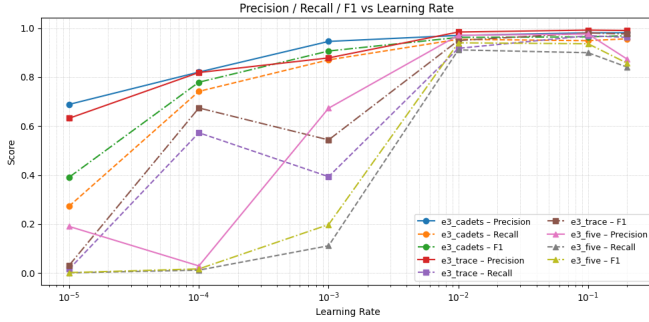


Fig. 8. Learning rate adjustment and effects in # E3 dataset.

G. Ablation Study on the Key Modules

To study the necessity of each portion of our approach, we replace the TraceCluster modules with other relative methods to observe how the model performance degrades.

We conduct ablation experiments as follows. First, we evaluate the detection efficiency when SAN is replaced by Cluster-GCN or GraphSAGE used in ThreaTrace. Next, we evaluate the detection efficiency of model variants where the attention mechanism or the clustering mechanism is removed. Finally, we evaluate the necessity of a designed weighting function, by removing weight setting and by setting the balanced sample weights.

We conducted experiments on three scenarios from the TC # E3 dataset. The experimental results (Table VIII) reveal significant performance differences.

Based on the analysis of experimental results, the SAN demonstrates the most stable performance across various scenarios, excelling in handling complex graph structures and fine-grained features. GraphSAGE follows closely as a strong alternative, particularly benefiting from its feature aggregation capability. In contrast, the performance of the GCN model declines as the complexity of the scenarios increases, reflecting its limitations in feature modelling. Without graph partitioning, the detection performance degrades significantly, indicating that subgraph clustering for feature extraction is essential. Adaptive category weighting mechanism enables TraceCluster to achieve superior detection performance, since the model is highly sensitive to correct class recognition.

To validate the feature extraction method we propose, we compare it with the node embedding method based on node types. The One-Hot encoding method [39] is employed for node embedding. It converts node types into feature vectors, focusing mainly on node type information while neglecting the interactions between the node and its neighbors. The experimental results, as shown in Table VIII, demonstrate that our feature extraction method, by providing more structural information, can better capture the interactions between nodes, thereby improving the ability to detect attack behaviors.

TABLE VIII
THE IMPACT OF DIFFERENT MODULES IN TRACECLUSTER

Dataset/Scene	Model	Precision	Recall	F1-score
# E3 Trace	Cluster-GCN	0.85	0.66	0.74
	GraphSAGE	0.97	0.88	0.92
	w/o Partition	0.85	0.66	0.74
	w/o Attention	0.87	0.75	0.81
	w/o Weight	0.93	0.88	0.90
	Balanced weight	0.92	0.92	0.92
	Node Embedding	0.87	0.65	0.74
	TraceCluster	0.98	0.95	0.96
# E3 Cadets	Cluster-GCN	0.91	0.86	0.88
	GraphSAGE	0.95	0.93	0.94
	w/o Partition	0.81	0.79	0.80
	w/o Attention	0.93	0.89	0.91
	w/o Weight	0.73	0.76	0.75
	Balanced weight	0.89	0.90	0.90
	Node Embedding	0.88	0.61	0.72
	TraceCluster	0.97	0.94	0.95
# E3 FiveDirections	Cluster-GCN	0.84	0.72	0.77
	GraphSAGE	0.93	0.83	0.88
	w/o Partition	0.76	0.51	0.61
	w/o Attention	0.87	0.75	0.81
	w/o Weight	0.92	0.86	0.89
	Balanced weight	0.90	0.82	0.86
	Node Embedding	0.83	0.70	0.76
	TraceCluster	0.94	0.90	0.92

TABLE IX
COMPARISON OF EXPERIMENTAL RESULTS OF CLASS WEIGHT FUNCTION, SMOTE METHOD, AND FOCAL LOSS

Dataset/Scene	Model	Precision	Recall	F1-score
# E3 Trace	Weight	0.98	0.95	0.96
	SMOTE	0.97	0.93	0.95
	Focal Loss	0.96	0.97	0.96
# E3 Cadets	Weight	0.97	0.94	0.95
	SMOTE	0.95	0.92	0.93
	Focal Loss	0.92	0.93	0.93
# E3 FiveDirections	Weight	0.94	0.90	0.92
	SMOTE	0.90	0.86	0.88
	Focal Loss	0.91	0.88	0.90
Powershell Empire	Weight	0.89	0.84	0.86
	SMOTE	0.88	0.85	0.86
	Focal Loss	0.87	0.84	0.85
Malicious Upgrade	Weight	0.92	0.97	0.90
	SMOTE	0.91	0.95	0.93
	Focal Loss	0.90	0.93	0.91

H. Study of Different Catagory Weighting Methods

In this experiment, we compare the performance of the Class Weight function, SMOTE method [40], and Focal Loss [41] across three scenes in the # E3 dataset and two scenes in the OpTC dataset. As shown in Table IX, the Class Weight function demonstrates better performance in terms of both precision and recall across the three scenes. For example, in the # E3 Trace scene, the Class Weight function achieves a precision of 0.98 and recall of 0.95, outperforming SMOTE (precision 0.97, recall 0.93). Although SMOTE shows a slight advantage in recall, the Class Weight function generally yields more stable and accurate results. In addition, while

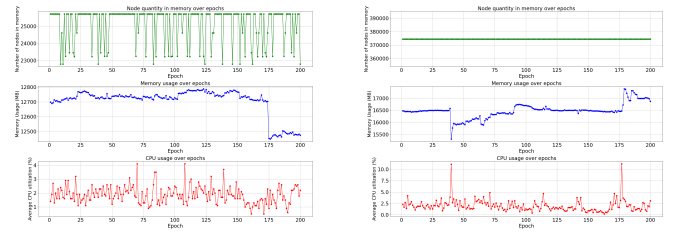


Fig. 9. Performance comparison of models with and without subgraph partitioning.

Focal Loss performs slightly lower than SMOTE in recall, it shows similar performance to the Class Weight function in terms of precision and F1-score, making it a competitive alternative. Overall, the Class Weight function shows superior performance in terms of classification stability, making it suitable for tasks requiring higher precision.

I. Resource Consumption During TraceCluster Execution

In this section, we primarily discuss the impact of subgraph partitioning on the efficiency of the model. The evaluation focuses on resource consumption, including the number of nodes before and after subgraph partitioning, memory usage, and CPU utilization during the training process. We use the # E3 FiveDirections dataset, and the experimental results are as follows. Figure 9 shows the performance comparison of models with and without subgraph partitioning.

By applying subgraph partitioning, the average memory consumption decreases from 16,500MB to 12,700MB, representing a reduction of approximately 24%. This effectively alleviated memory pressure during large-scale graph computation. Without subgraph partitioning, CPU utilization exhibits significant fluctuations and peaks, leading to heavier computational loads for individual tasks. However, after applying subgraph partitioning, the computational tasks become more distributed, significantly reducing the workload. At the same time, CPU utilization becomes lower and more stable, decreasing from a peak of 5% to 3%, a reduction of 40%.

According to our experimental results, we consider that subgraph partitioning not only effectively reduces memory usage and CPU load but also significantly improves the efficiency and stability of large-scale graph computations. Subgraph partitioning enables a more balanced allocation of computational resources, reducing resource bottlenecks and task imbalance. This provides a more efficient and scalable solution for training large-scale graph neural networks.

J. Analysis of Real-World Enterprise Logs

To evaluate the effectiveness of TraceCluster in a real-world enterprise environment, we conduct an open-world evaluation on an anonymized cyber range. The range serves the network security lifecycle management system of a Supercomputing Center. It supports the center's requirements for network security testing, attack-defense exercises, research validation, and professional training.

The range consists of 17 servers, each running CentOS 7.9 with Intel Xeon Gold 5218 processors (2×16 cores), 512GB of memory, and over 8TB of disk space. The evaluation spans 8 days, with the first 6 days dedicated to collecting normal behavior data and establishing a baseline model, while the final 2 days involve injecting known attack scenarios, including privilege escalation by exploiting the vulnerability CVE-2021-4034 and CVE-2021-3156, and remote command execution by CVE-2017-9791, for detection and evaluation. System logs and process behavior data are collected using CamFlow to build the baseline model for normal behavior. Approximately 150GB of relevant log data are produced per server per day. The model's performance is then evaluated based on key metrics, including Precision, Recall, False Positive Rate, and F1-score, providing a comprehensive assessment of its effectiveness.

Results and Analysis: We evaluate TraceCluster's performance in attack detection, and the results show an accuracy of 68% and a recall of 50%. Although TraceCluster performs reasonably well in a controlled laboratory environment, its effectiveness in a real production environment falls short of expectations. The accuracy of 68% indicates that most predictions are correct; however, due to data imbalance, the model tends to favor normal behavior and fails to effectively detect attack behaviors. The Recall of 50% means it misses half of the attack events, primarily due to the variability of attack patterns across different environments, which hinders its ability to adapt to these changes. The F1-score of 0.576 reflects TraceCluster's imbalance between accuracy and recall, which impacts overall performance.

The diversity of attack patterns causes the same attack to manifest differently across different operating systems and hardware, affecting the recall rate. Additionally, noise and interference in real environments, such as system updates and network fluctuations, are often misclassified as attacks, increasing the detection difficulty. The model struggles to adapt to the constantly changing attack strategies in production environments, while the complexity of normal behavior and the instability of data quality further limit its detection performance and generalization ability.

VII. DISCUSSION AND LIMITATION

A. Real-Time Detection Constraints

Although TraceCluster significantly reduces inference time compared with FLASH, achieving real-time APT detection in large-scale distributed systems still imposes substantial computational pressure. Partitioning, training, and processing continuously evolving provenance graphs introduce additional overhead, which becomes particularly evident in highly dynamic environments with high event throughput. In this work, we have investigated incremental and streaming variants of TraceCluster, and observed that introducing new nodes or edges after subgraph partitioning disrupts existing partitions and the underlying pipeline assumptions, necessitating complete repartitioning. Experimental results show that such incremental subgraph partitioning and model update mechanisms substantially degrade detection efficiency. In future

work, we plan to explore dynamic subgraph update mechanisms to enhance TraceCluster's adaptability to continuously incoming data streams.

B. Interpretability and Analyst Usability

Despite its strong detection performance, TraceCluster, like many graph neural network-based systems, operates as a black box. The subgraph attention mechanism provides implicit interpretability at the feature level, but it does not directly yield causal explanations or reconstruct complete attack chains. Enhancing interpretability, for instance, through attention visualization, provenance-based reasoning, or graph-level explanation modules, would increase the practical usability of TraceCluster in security operations.

C. Assumption of Complete and Trustworthy Provenance Collection

APT actors are capable of obfuscating collected log data through malicious tactics such as log tampering, event dropping, and inserting benign-looking edges into the provenance graph. In such adversarial settings, not only our proposed model but also a majority of the aforementioned state-of-the-art methods are likely to suffer from performance degradation or even complete failure. Considering this gap, we posit that strategies such as edge pruning and node aggregation hold substantial promise for mitigating the adverse impacts of untrustworthy log data. Exploring the design and optimization of these countermeasures will therefore constitute a key direction of our future research efforts.

VIII. CONCLUSION

The proposed TraceCluster demonstrates significant advancements in addressing APT attack detection on large-scale provenance graphs. By integrating the clustering-based subgraph attention network, class weighting mechanisms, and graph partitioning techniques, TraceCluster significantly improves detection efficiency and accuracy.

In addition, TraceCluster reduces computational overhead through subgraph partitioning, allowing large-scale data processing while maintaining effective detection of attack behaviors. Its lightweight design yields significant efficiency gains, outperforming faster lightweight node-level detectors by 37-fold and 3-fold faster inference on the OpTC dataset.

Future work will focus on improving TraceCluster in three aspects. First, we plan to enhance scalability to support real-time detection in dynamic environments. Second, we will improve model interpretability to provide clearer and more actionable insights for security analysts. Finally, we will investigate the problem of APT detection under the conditions of unreliable traceability data, training data drift, and structural changes.

REFERENCES

- [1] M. Ur Rehman, H. Ahmadi, and W. Ul Hassan, "Flash: A comprehensive approach to intrusion detection via provenance graph representation learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 3552–3570.

- [2] D. Beyer and A. Fararooy, "A simple and effective measure for complex low-level dependencies," in *Proc. IEEE 18th Int. Conf. Program Comprehension*, Jun. 2010, pp. 80–83.
- [3] I. Stelios, P. Kotzanikolaou, M. Psarakis, C. Alcaraz, and J. Lopez, "A survey of IoT-enabled cyberattacks: Assessing attack paths to critical infrastructures and services," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3453–3495, 4th Quart., 2018.
- [4] D. Shan, X. Du, W. Wang, N. Wang, and A. Liu, "KPI-HGNN: Key provenance identification based on a heterogeneous graph neural network for big data access control," *Inf. Sci.*, vol. 659, Feb. 2024, Art. no. 120059.
- [5] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. N. Venkatakrishnan, "POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1795–1812.
- [6] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 1137–1152.
- [7] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.
- [8] S. Wang et al., "THREATTRACE: Detecting and tracing host-based threats in node level through provenance graph learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3972–3987, 2022.
- [9] R. Guo, L. Cheng, J. Li, P. R. Hahn, and H. Liu, "A survey of learning causality with data: Problems and methods," *ACM Comput. Surveys*, vol. 53, no. 4, pp. 1–37, Jul. 2021.
- [10] H. Studiawan, F. Sohel, and C. Payne, "Anomaly detection in operating system logs with deep learning-based sentiment analysis," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2136–2148, Sep. 2021.
- [11] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1777–1794.
- [12] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2019, pp. 807–817.
- [13] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 1035–1044.
- [14] Z. Cheng et al., "Kairos: Practical intrusion detection and investigation using whole-system provenance," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2024, pp. 3533–3551.
- [15] A. Aly, S. Iqbal, A. Youssef, and E. Mansour, "MEGR-APT: A memory-efficient APT hunting system based on attack representation learning," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 5257–5271, 2024.
- [16] B. Jiang et al., "Orthrus: Achieving high quality of attribution in provenance-based intrusion detection systems," in *Proc. Secur. Symp. (USENIX Secur.)*, 2025, pp. 7173–7192.
- [17] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2016, pp. 1–14. [Online]. Available: <https://openreview.net/forum?id=SJU4ayYgl>
- [18] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [19] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2008.
- [20] J. Chen, T. Ma, and C. Xiao, "FastGCN: Fast learning with graph convolutional networks via importance sampling," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–15.
- [21] J. Chen, J. Zhu, and L. Song, "Stochastic training of graph convolutional networks with variance reduction," in *Proc. 32nd Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1–11.
- [22] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 257–266.
- [23] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1285–1298.
- [24] Y. Shen, E. Mariconti, P.-A. Vervier, and G. Stringhini, "Tiresias: Predicting security events through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2018, pp. 592–605.
- [25] Y. Yu, X. Si, C. Hu, and J. Zhang, "A review of recurrent neural networks: LSTM cells and network architectures," *Neural Comput.*, vol. 31, no. 7, pp. 1235–1270, Jul. 2019.
- [26] B. Xia, J. Yin, J. Xu, and Y. Li, "LogGAN: A sequence-based generative adversarial network for anomaly detection based on system logs," in *Science of Cyber Security*. Cham, Switzerland: Springer, 2019, pp. 61–76.
- [27] T. Li, X. Liu, W. Qiao, X. Zhu, Y. Shen, and J. Ma, "T-trace: Constructing the APTs provenance graphs through multiple syslogs correlation," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1179–1195, May 2024.
- [28] M. Togninalli, E. Ghisu, F. Llinares-López, B. Rieck, and K. Borgwardt, "Wasserstein Weisfeiler–Lehman graph kernels," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 6439–6449.
- [29] J. Zhou, X. Zhang, Y. Zhao, W. Zhao, and Y. Li, "Provg-searcher: A graph representation learning approach for efficient provenance graph search," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 5, pp. 887–900, May 2020.
- [30] F. Dong et al., "DISTDET: A cost-effective distributed cyber threat detection system," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 6575–6592.
- [31] J. Zengy et al., "SHADEWATCHER: Recommendation-guided cyber threat analysis using system audit records," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 489–506.
- [32] S. Li et al., "NODLINK: An online system for fine-grained APT attack detection and investigation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024, pp. 1–18.
- [33] G. Karypis and V. Kumar, "Multilevel algorithms for multi-constraint graph partitioning," in *Proc. IEEE/ACM SC98 Conf.*, Nov. 1998, p. 28.
- [34] T. Pasquier et al., "Practical whole-system provenance capture," in *Proc. Symp. Cloud Comput.*, Sep. 2017, pp. 405–418.
- [35] B. W. Kernighan and S. Lin, "An efficient heuristic procedure for partitioning graphs," *Bell Syst. Tech. J.*, vol. 49, no. 2, pp. 291–307, Feb. 1970.
- [36] L.-T. Liu, M.-T. Kuo, S.-C. Huang, and C.-K. Cheng, "A gradient method on the initial partition of fiduccia-mattheyses algorithm," in *Proc. IEEE Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 1995, pp. 229–234.
- [37] B. Jacob, P. Larson, and B. H. Leita, "Systemtap: Instrumenting the Linux kernel for analyzing performance and functional problems," IBM Redpaper, Jan. 2025. [Online]. Available: <http://www.redbooks.ibm.com/abstracts/redp4469.html>
- [38] A. D. Keromytis. (2018). *Transparent Computing Engagement 3 Data Release*. Accessed: Aug. 10, 2025. [Online]. Available: <https://github.com/darpa-i2o/Transparent-Computing/blob/master/README3.md>
- [39] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," *J. Mach. Learn. Res.*, vol. 3, pp. 1137–1155, Feb. 2003.
- [40] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [41] T. Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollr, "Focal loss for dense object detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, pp. 318–327, 2020.